

Introduction to C#

A Beginner-Friendly Guide to Modern Programming with C#

Author: Luv, Bill

Table of Contents

1. Introduction to C#
2. Setting Up the Development Environment
3. Your First C# Program
4. Variables and Data Types
5. Operators in C#
6. Control Flow (if, switch, loops)
7. Methods and Functions
8. Object-Oriented Programming (OOP)
9. Classes and Objects
10. Inheritance and Polymorphism
11. Encapsulation and Abstraction
12. Arrays and Collections
13. Exception Handling
14. File Handling
15. Introduction to LINQ
16. Building Simple Console Applications
17. Best Practices in C#
18. Conclusion

Introduction

C# (pronounced “C-Sharp”) is a modern, powerful programming language developed by Microsoft. It is widely used for building:

- desktop applications
- web applications (ASP.NET)
- mobile apps (Xamarin / .NET MAUI)
- games (Unity)
- enterprise software

This ebook is designed for beginners who want to learn programming using C# from scratch and build a strong foundation in software development.

Chapter 1 — Introduction to C#

C# is an object-oriented programming language built on the .NET framework.

Why Learn C#?

- Easy to learn for beginners
- Powerful and scalable
- Used in professional environments
- Strong community and support

- Works across platforms with .NET

What You Can Build with C#

- Games using Unity
- Web APIs and websites
- Desktop applications
- Mobile apps
- Cloud applications

Chapter 2 — Setting Up the Development Environment

To start programming in C#, you need:

- .NET SDK
- Visual Studio or Visual Studio Code

Download .NET

[.NET Official Website](https://dotnet.microsoft.com)

Recommended IDE

- Visual Studio Community (best for beginners)
- Visual Studio Code (lightweight option)

Chapter 3 — Your First C# Program

```
using System;
```

```
class Program
```

```
{  
    static void Main()  
    {  
        Console.WriteLine("Hello, World!");  
    }  
}
```

Explanation

- Main() is the entry point
- Console.WriteLine() prints output

Chapter 4 — Variables and Data Types

Variables store data.

Example

```
int age = 25;  
string name = "Chris";  
double price = 10.5;  
bool isActive = true;
```

Common Data Types

Type	Description
int	Whole numbers
string	Text
double	Decimal numbers
bool	True/false

Chapter 5 — Operators in C#

Arithmetic Operators

```
int sum = 5 + 3;
```

Comparison Operators

```
bool result = 5 > 3;
```

Logical Operators

```
bool result = true && false;
```

Chapter 6 — Control Flow

If Statement

```
int age = 18;
```

```
if (age >= 18)
{
    Console.WriteLine(" Adult");
}
```

Switch Statement

```
int day = 1;

switch(day)
{
    case 1:
        Console.WriteLine(" Monday");
        break;
}
```

Loops

For Loop

```
for(int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
}
```

Chapter 7 — Methods and Functions

```
static void SayHello()
{
    Console.WriteLine(" Hello");
}
```

Method with Parameters

```
static int Add(int a, int b)
{
    return a + b;
}
```

Chapter 8 — Object-Oriented Programming (OOP)

C# is based on OOP principles.

Core Principles

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

Chapter 9 — Classes and Objects

Class Example

```
class Person
{
    public string name;
    public int age;
}
```

Object Creation

```
Person p = new Person();
p.name = "Lena";
```

Chapter 10 — Inheritance and Polymorphism

Inheritance

```
class Animal
{
    public void Eat()
    {
        Console.WriteLine(" Eating...");
    }
}
```

```
class Dog : Animal
{
}
```

Polymorphism

```
class Animal
{
    public virtual void Sound()
    {
        Console.WriteLine(" Animal sound" );
    }
}
```

Chapter 11 — Encapsulation and Abstraction

Encapsulation

```
class BankAccount
{
    private double balance;
}
```

Abstraction

```
abstract class Shape
{
    public abstract void Draw();
}
```

Chapter 12 — Arrays and Collections

Array Example

```
int[] numbers = {1, 2, 3};
```

List Example

```
List<string> names = new List<string>();
names.Add("Chris");
```

Chapter 13 — Exception Handling

```
try
{
    int result = 10 / 0;
}
catch(Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

Chapter 14 — File Handling

```
using System.IO;
```

```
File.WriteAllText("file.txt", "Hello C#");
```

Chapter 15 — Introduction to LINQ

LINQ helps query data easily.

Example

```
var result = numbers.Where(n => n > 2);
```

Chapter 16 — Building Simple Console Applications

Example: Calculator

```
int a = 5;  
int b = 3;  
Console.WriteLine(a + b);
```

Chapter 17 — Best Practices in C#

- Use meaningful variable names
- Write clean and readable code
- Avoid duplication
- Follow OOP principles
- Handle exceptions properly

Chapter 18 — Conclusion

C# is a powerful and versatile programming language that opens doors to many areas of software development. By learning the fundamentals covered in this ebook, beginners can build a strong foundation and progress toward advanced development in web, desktop, and game programming.

Final Advice

- Practice coding every day
- Build small projects
- Understand OOP deeply
- Learn .NET ecosystem
- Keep improving step by step

End

Introduction to C# — Beginner Programming Guide